

Chapter 1

Sideband Verification Environment

1.1 Introduction and Verification Scope

The Sideband interface is a critical component of the system, responsible for initialization, configuration, and out-of-band message passing. The primary objective of this verification environment is to ensure absolute protocol compliance, data integrity, and robust error handling across the Module Die Interface (MDI). This chapter details the transaction-level UVM architecture, the modular agent topology, dynamic reset handling, the self-checking infrastructure, and the coverage strategy used to verify the Sideband block.

1.2 UVM Environment Structure

To achieve a scalable and reusable verification environment, the architecture strictly adheres to the Universal Verification Methodology (UVM) hierarchy.

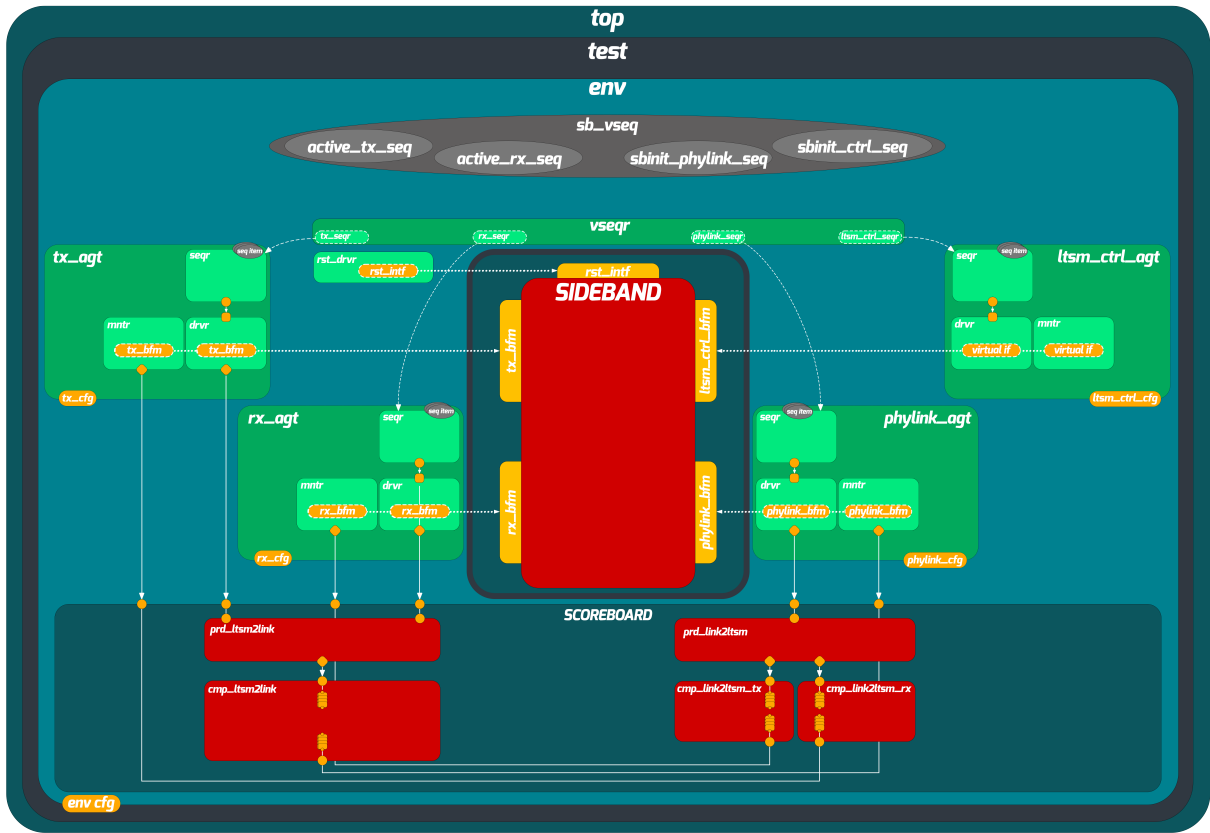


Figure 1.1: Hierarchical block diagram of the Sideband UVM Environment.

As illustrated in Figure 1.1, the top-level simulation module encapsulates both the Design Under Test (DUT) and the physical Bus Functional Models (BFMs). Inside the UVM domain, the `uvm_test_top` instantiates the top-level `sb_env`. The environment acts as the central hub, containing:

- A Virtual Sequencer (`vseqr`) to orchestrate multi-interface traffic.
- A dedicated Reset Driver (`rst_drvr`) to handle asynchronous system resets.
- Four currently implemented UVM Agents for localized interface driving and monitoring, with a fifth D2D RDI agent planned for a future revision.
- A centralized Scoreboard (`sb`) for end-to-end data integrity checking.

1.3 Testbench Architecture

Designing a scalable and maintainable testbench requires careful consideration of the DUT boundaries and interface complexity.

1.3.1 Agent Topology

The Sideband interface presents a high degree of complexity, encompassing more than 35 discrete physical signals. To handle this complexity, the environment was architected around five distinct UVM agents rather than a single, monolithic interface agent. At the current stage of development, however, only four agents are implemented because the D2D RDI interface is not yet supported.

Instead of relying on general modularity arguments alone, this approach was driven by the physical realities of the Sideband block. Each agent operates independently and acts as a proxy for a specific external block communicating with the Sideband. The current implementation includes the following agents:

- LTSM TX Agent (`tx_agt`)
- LTSM RX Agent (`rx_agt`)
- Remote Link Agent (`phylink_agt`)
- LTSM Control Agent (`ltsm_ctrl_agt`)

The D2D RDI Agent (`rdi_agt`) remains part of the planned architecture, but it is intentionally omitted from the present environment until RDI support is added.

This topology offers strong practical benefits. During development, it allowed the verification work to focus on one independent functionality at a time when building drivers and monitors. In terms of stimulus, the same modularity empowers virtual sequences to manage multiple agents in parallel and create complex concurrent scenarios with limited overhead. Debugging is also improved because protocol violations can be traced back directly to the interface-specific component that observed them.

1.3.2 Bus Functional Models (BFMs)

To keep the UVM drivers and monitors abstracted from physical timing details, Bus Functional Models (BFMs) were heavily utilized. The BFMs encapsulate pin-level toggling, clock-edge alignment, serialization details, and intra-packet delays. By handling these physical-layer constraints inside the SystemVerilog interfaces, the UVM components remain reusable and focused on transaction intent rather than waveform construction.

1.4 Dynamic Reset Testing Methodology

A robust verification environment must gracefully handle dynamic, mid-simulation resets without locking up or generating false errors. To achieve this, the environment implements the UVM phase-jumping framework detailed by Hunter et al. in [1].

The reset methodology combines both idle reset testing and active reset testing. Idle reset testing verifies that the environment can safely restart after a run has naturally drained. Once the stimulus has completed, the DUT becomes quiescent, and the scoreboard no longer holds pending transactions, the base test redirects execution back to the reset phases and starts another run. This checks that the environment can re-enter reset

from a clean state without leaving stale transactions, stale objections, or stale predictor data behind.

Active reset testing targets the more stressful case in which reset is asserted while transactions are still flowing through the DUT. In the base test, the main virtual sequence starts normally, and a second thread waits for a randomized delay before forcing a jump back to the reset phases. In this way, reset can interrupt ongoing local traffic, incoming remote traffic, or internal routing activity. This style of testing is particularly important because it exposes bugs that do not appear in clean start-to-finish runs, such as partially updated internal state, broken arbitration recovery, or stale data remaining inside verification components.

To make phase jumping safe, the environment itself was built to be reset-aware. The virtual sequencer stops outstanding sequences before reset begins, monitor threads terminate and restart around reset boundaries, comparator FIFOs are flushed before the next run starts, and predictor-side queues are cleared so that transactions from the old run cannot be matched against transactions from the new run. This follows the reset philosophy described in [1]: phase jumps are effective only when all stateful verification components know how to recover cleanly.

1.5 Assertion-Based Verification (ABV)

To mathematically guarantee the Sideband Initialization (SBINIT) protocol, Assertion-Based Verification (ABV) was integrated directly at the interface level. SystemVerilog Assertions (SVA) were written to capture critical handshakes, ensure negative-edge data sampling, and detect zero-time glitches on clock lines.

The assertion set focuses on observable SBINIT behavior at the interface level rather than on internal RTL implementation details. One group of properties checks correct initialization-pattern generation and ensures that readiness is only reached after the required training behavior has been completed. A second group checks the mandatory low-idle windows on both the data and clock lines during the off portion of the SBINIT cycling. A third checker validates reset cleanliness by ensuring that key outputs return to their reset values when reset is asserted. In addition, a dedicated glitch detector watches the transmitted clock and flags zero-time toggles that would correspond to physically invalid waveform behavior. These assertions complement the scoreboard by proving temporal protocol rules that are difficult to verify through transaction comparison alone.

Because assertions act as independent checkers, it is critical that they themselves be trusted. Every assertion was therefore verified using targeted SVA unit tests to ensure that no false passes or missed failures could corrupt the top-level regression results.

1.6 Checking Mechanisms (The Scoreboard)

All four currently implemented agents actively communicate with the scoreboard. To prevent a large explosion of TLM port connections and maintain data routing clarity, a highly structured scoreboard architecture was developed. When RDI support is added, the same scoreboard architecture is intended to scale to that fifth agent as well.

1.6.1 Predictor and Comparator Separation

The architecture heavily borrows from established UVM scoreboard design patterns, specifically advocating for the strict separation of the Predictor and the Comparator [2]. The prediction logic itself was split into two independent predictors to handle the bi-directional nature of the Sideband:

1. **prd_ltsm2link**: Models the outbound direction, predicting the output to the remote link based on traffic from the currently supported LTSM TX and RX drivers, and is structured to later incorporate the RDI driver once that interface is supported.
2. **prd_link2ltsm**: Models the inbound direction. It takes a single input from the Remote Link driver and predicts the routing to the currently active local comparators based on the decoded direction of the incoming message.

The three active comparators each check one observable verification path. The **cmp_ltsm2link** comparator verifies the outbound direction by comparing the predictor output against the phylink transaction reconstructed by the remote-link monitor at the DUT boundary. The **cmp_link2ltsm_tx** comparator checks the inbound direction for messages that are expected to emerge on the local TX-facing interface after being received from the link and translated by the DUT. Likewise, the **cmp_link2ltsm_rx** comparator checks inbound traffic that should emerge on the local RX-facing interface. Although each comparator targets a different path, they all follow the same checking philosophy: wait for an expected transaction, allow a bounded DUT latency window, report a timeout if the DUT drops the transaction, and otherwise compare the predicted and observed transactions field by field.

1.6.2 Cross-Die Message Translation

During the development of the **prd_link2ltsm** predictor, a critical protocol nuance was identified regarding cross-die message mapping. When the local Sideband receives a message from the remote die's TX path, it may need to forward it to the local RX path, and vice versa. However, the receiving local interface expects the encoding specific to its own domain rather than the original remote encoding.

To accurately model this behavior, the predictor was refined using two associative arrays acting as hardware look-up tables (LUTs): **tx2rx_enc_lut** and **rx2tx_enc_lut**. These tables translate the encodings before the expected transaction reaches the relevant comparator, ensuring that the predicted transaction reflects the same semantic message as seen by the local receiving interface.

1.7 Transaction Level Modeling (Sequence Items)

At the core of the verification environment is the transition from pin-level signals to an abstract, object-oriented view using UVM sequence items. The agents serve as the translation layer, converting abstract sequences into physical signals and vice versa.

ltsm_seq_item models the local, parallel-facing view of the Sideband protocol. It captures the message direction, the local encoding or message classification, the optional data payload, and the sideband information field without embedding pin-level timing. This makes it the natural transaction abstraction for the TX and RX local interfaces.

phylink_seq_item, by contrast, models the remote serialized view of the protocol. It can represent either SBINIT behavior or ACTIVE message traffic on the physical link. For ACTIVE traffic, it carries the logical message fields such as fullcode, opcode, source and destination identifiers, information field, payload, and parity status. For SBINIT, it represents the initialization pattern behavior and the timing-oriented attributes needed by the remote-link BFM.

These two transaction abstractions are connected to the signal-level interfaces by the UVM agents. On the stimulus side, the TX and RX drivers translate **ltsm_seq_item** objects into local handshake, encoding, information, and payload activity, while the phylink

driver translates `phylink_seq_item` objects into serialized SBINIT or ACTIVE traffic on the remote link pins. On the checking side, the monitors perform the inverse operation and reconstruct the same transaction-level objects from observed DUT behavior. This allows the environment to reason about Sideband traffic in a transaction-oriented form while still driving and sampling the real interfaces.

To maximize readability and constrainability, the protocol model was organized using strongly typed enumerations, logical message structures, and associative lookup tables rather than anonymous bit-vectors. The shared package defines the supported TX and RX encodings, the protocol fullcodes and opcodes, the source and destination identifiers, and a canonical message structure that groups the logical fields of a Sideband packet into one object. The same package also stores associative arrays containing the supported message templates, the currently recognized unsupported fullcodes, and the cross-die translation tables. Utility functions then bridge between these abstractions and the wire representation by packing and unpacking messages, extracting message fields, deriving message type information, and recalculating parity when needed.

1.7.1 `ltsm_seq_item` Randomization Strategy

The constrained-random strategy for `ltsm_seq_item` is direction-aware. Before randomization, the item is configured to represent either TX-originated or RX-originated traffic, and only the corresponding encoding space is then explored. This prevents illegal combinations that do not belong to the selected source. Additional constraints preserve protocol realism by forcing header-only message classes to carry no payload, bounding the wait interval between transactions, and biasing the stimulus toward corner values in the information and data fields. As a result, the item can generate both ordinary message traffic and useful boundary-value cases while still remaining aligned with the legal local protocol view.

1.7.2 `phylink_seq_item` Randomization Strategy

The randomization strategy for `phylink_seq_item` is mode-aware. When configured for SBINIT, the item focuses on initialization behavior such as training-pattern generation, timing skew, and controlled corruption used to stress the initialization detector. When configured for ACTIVE traffic, it primarily draws from the set of known protocol message templates so that most randomized traffic remains meaningful and checkable. At the same time, it can deliberately generate unsupported or corrupted traffic to verify robustness against invalid cases. After the message form is selected, parity is derived from the logical fields and optional corruption may then be introduced in a controlled way, allowing the same item to cover both nominal protocol traffic and negative testing scenarios.

1.8 Test Strategy and Stimulus Scenarios

Leveraging the current 4-agent implementation of the planned 5-agent architecture, the stimulus generation strategy relies on virtual sequences acting as orchestrators across the independent agent sequencers.

1.8.1 Base Test Infrastructure

All top-level tests inherit from a common base test. This layer creates the environment, publishes the required virtual interfaces, constructs the shared environment configuration, and launches the selected virtual sequence on the virtual sequencer. Because each test reuses this infrastructure, the derived test classes only need to choose the desired virtual

sequence, enable any required operating mode, and define a suitable timeout. The same base layer also embeds the reset methodology described earlier, which means each major test automatically benefits from the same reset-aware framework without duplicating reset-control logic.

1.8.2 Sanity Test

The `sb_sanity_test` is the initial bring-up test for the environment. Its virtual sequence first coordinates both sides of SBINIT: the control-side sequence starts the initialization flow, while the remote-link sequence drives the expected incoming training behavior. After initialization completes successfully, the test issues a small set of directed ACTIVE transactions across the local TX path, the local RX path, and the remote link. The purpose of this test is not exhaustiveness, but rather to prove that the environment can bring the DUT out of reset, complete initialization, drive representative traffic on each supported interface, and obtain clean scoreboard and assertion behavior.

1.8.3 Send-All Test

The `sb_sendall_test` reuses the same initialization flow as the sanity test, but replaces the short directed ACTIVE phase with an exhaustive protocol sweep. Its virtual sequence launches dedicated TX, RX, and phylink sequences that systematically walk through the supported message definitions stored in the environment model. Rather than relying on random generation, these sequences intentionally exercise each supported encoding and its corresponding protocol form at least once. This makes the test especially effective at exposing mismatches between the verification model and the RTL specification, particularly in areas such as encoding values, opcode selection, and message classification.

1.8.4 Random Test

The `sb_rand_test` extends the environment from directed protocol checking into high-volume constrained-random stress. After bringing the DUT through initialization, it drives randomized local TX traffic, randomized local RX traffic, and a larger volume of randomized phylink traffic. Because the underlying sequence items are constrained using the strategies described earlier, the test explores a wide legal space while still injecting carefully selected invalid or corrupted cases. This makes the random test effective both for deepening coverage and for revealing subtle data-handling or parity-related issues that are unlikely to surface in purely directed testing.

1.8.5 Concurrent Test

The `sb_conc_test` targets the interaction between simultaneous traffic sources. After the usual initialization phase, its virtual sequence launches TX, RX, and phylink activity in parallel so that the DUT must handle concurrent requests instead of isolated streams. To model the LTSM more faithfully, the local TX and RX paths are run in reactive mode. In this mode, the output monitors forward observed DUT responses back to the corresponding sequencers, and each local sequence waits for the response to the current message before issuing the next one. This creates a closed-loop stimulus style that mirrors request-response dependence in the real system and is particularly effective at exposing arbitration and reset-recovery problems that only appear when multiple paths are active together.

1.9 Unit Testing the Verification Components

A robust testbench cannot be trusted if the testbench itself is flawed. To ensure the integrity of the UVM components, SVUnit was integrated into the development workflow [3]. By unit testing the predictors, comparators, SVA properties, and BFMs in strict isolation prior to top-level integration, the testbench is proven stable, ensuring that any failures found during top-level regressions are legitimate RTL bugs.

1.10 Coverage Strategy

Coverage was collected in two complementary forms. Code coverage was used to measure how thoroughly the regression exercised the implemented RTL structure, while functional coverage was used to measure whether the intended protocol behaviors and corner cases had actually been explored. The functional coverage model is currently centered on the phylink domain, where it tracks valid and invalid messages, supported and unsupported message categories, opcode classes, source and destination identifiers, payload extremes, and important cross combinations between routing, validity, and data conditions.

To keep the reported code coverage meaningful, a small set of exclusions was applied in the simulation scripts. These exclusions remove assertion code from RTL code-coverage accounting and also exclude toggle points that are either fixed by protocol definition or intentionally unreachable under the current legal stimulus strategy, such as hard-wired protocol fields and selected backpressure conditions. The numerical coverage results themselves are presented later in the Results section.

1.11 Results and Conclusion

The integration of the environment and the execution of the major tests immediately yielded a high Return on Investment (ROI), uncovering several critical RTL and model-alignment issues while also validating the core checking infrastructure of the Sideband environment.

1.11.1 Sanity Test Evidence

The `sb_sanity_test` produced two complementary forms of evidence. First, the assertion waveform confirmed that the full set of SBINIT assertions remained satisfied throughout the sanity sequence. Second, the scoreboard logs showed successful predictor-to-monitor matches for both outbound phylink traffic and inbound link-to-LTSM routing.

Representative scoreboard matches from the sanity-test log are summarized as follows:

- A successful `LTSM2LINK_CMP` comparison was recorded for the initialization path, with the predictor output matching the RTL monitor transaction field for field.
- A successful outbound `ACTIVE`-message comparison confirmed correct handling of a payload-carrying transfer, including its information field and data payload.
- A successful inbound comparison confirmed correct routing and encoding translation after link reception.
- The monitor summary further showed activity on the supported interfaces, confirming that the test exercised both the local paths and the remote link path.

The comparator summary at the end of the run showed that the active comparator instances completed with zero mismatches, while unsupported or currently inactive

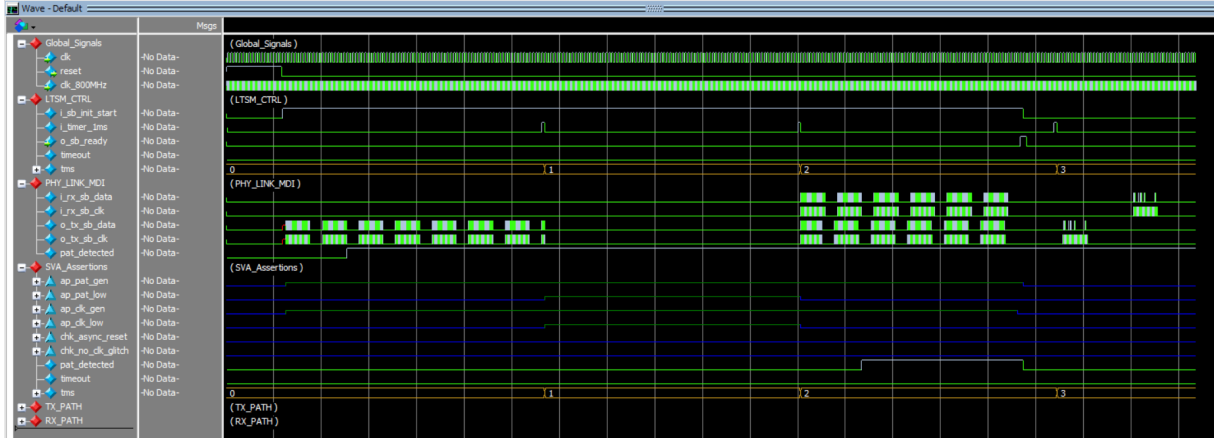


Figure 1.2: Waveform snapshot showing the SBINIT sanity-sequence assertions passing during the sanity test.

paths reported zero executed vectors, which is consistent with the present environment configuration in which the RDI interface is not yet supported.

Initialization Issues Discovered:

- Data was toggling synchronously with the clock rather than at the required data rate.
- SVA caught zero-time glitches occurring on the clock line.
- The RTL was incorrectly sampling data on the positive edge, violating the UCle requirement for negative-edge sampling.
- Corrupted, extraneous data was transmitted on the receive-side data line before the actual initialization pattern began.

Message Transmission Issues Discovered:

- Negative-edge sampling failures persisted during ACTIVE message transmission.
- A power and gating issue was identified where the clock toggled continuously instead of only toggling when valid data was present to send.

1.11.2 Send-All Test Findings

The `sb_sendall_test` was primarily used as a specification-alignment test. Because it systematically exercised the supported message space instead of sampling it randomly, it quickly exposed mismatches between the verification environment model and the RTL implementation. The dominant class of issues found by this test involved encoding and opcode mismatches. Some of these mismatches were traced back to mistakes in the environment model, while others were traced back to RTL defects. This result demonstrates the value of an exhaustive protocol sweep early in the verification cycle: it not only checks the DUT, but also validates that the verification model itself reflects the intended protocol definition accurately.

The following scoreboard log excerpt shows a representative mismatch caught during the `sb_sendall_test`. In this case, the predictor expected `End_Rx_Init_D_to_C_eye_sweep_resp`, while the RTL produced `End_Rx_Init_D_to_C_eye_sweep_req`, demonstrating an encoding-level specification mismatch on the outbound path.

```

# UVM_ERROR ../tb/scoreboard/sb_cmp_base.svh(125) @ 67632: uvm_test_top.env.sb_cmp_lt
# Transaction Mismatch Detected!
#
# -----
# EXPECTED TRANSACTION (From Predictor):
# -----
# -----
# Name                Type                Size  Value
# -----
# t_out                phylink_seq_item -      @3930
#   op_mode            operation_t        1      ACTIVE
#   pattern            integral          64     'hx
#   idle_ui_cnt        integral          32     'h0
#   fullcode           fullcode_t        16     End_Rx_Init_D_to_C_eye_sweep_resp
#   opcode             opcode_t          5      MSG_WO_DATA
#   srcid              srcid_t           3      SRC_PHY
#   dstid              dstid_t           3      DST_PHY
#   info               integral          16     'h0
#   data               integral          64     'h0
#   cp                 integral          1      'h1
#   dp                 integral          1      'h0
# -----
#
#
# -----
# ACTUAL TRANSACTION (From RTL Monitor):
# -----
# -----
# Name                Type                Size  Value
# -----
# _item               phylink_seq_item -      @3912
#   op_mode            operation_t        1      ACTIVE
#   pattern            integral          64     'hx
#   idle_ui_cnt        integral          32     'h0
#   fullcode           fullcode_t        16     End_Rx_Init_D_to_C_eye_sweep_req
#   opcode             opcode_t          5      MSG_WO_DATA
#   srcid              srcid_t           3      SRC_PHY
#   dstid              dstid_t           3      DST_PHY
#   info               integral          16     'h0
#   data               integral          64     'h0
#   cp                 integral          1      'h1
#   dp                 integral          1      'h0
# -----

```

1.11.3 Random Test Findings

The `sb_rand_test` exposed bugs that were harder to uncover with directed testing alone. One bug affected payload ordering, where the RTL placed the least significant 32 bits of the payload into the most significant 32-bit position and vice versa. A second bug

affected parity generation, where the control-parity calculation incorrectly included the data-parity contribution. A third issue appeared around unsupported fullcodes, specifically for unsupported cases whose hexadecimal representation contains A in the second digit, which triggered incorrect behavior in the RTL handling of invalid traffic. These findings highlight the importance of constrained-random stress in revealing subtle data-path and negative-case issues beyond simple message-definition mismatches.

1.11.4 Concurrent and Reset Test Findings

The `sb_conc_test`, combined with the active-reset mechanism in the base test, exposed a reset-recovery bug in the concurrent traffic logic. When a reset was applied while both TX and RX traffic were active, part of the RTL traffic-management logic did not return to a clean reset state. As a result, the post-reset prioritization of concurrent TX and RX messages followed an incorrect flow. This issue would have been difficult to observe in isolated directed testing because it depended on the interaction between concurrency, in-flight traffic, and a sudden asynchronous reset event.

1.11.5 Regression Results

After the directed and random debugging phases, the major stress tests were consolidated into regression. The regression included `sb_sendall_test`, `sb_rand_test`, and `sb_conc_test`, each executed in ten runs. All runs completed successfully. This regression campaign achieved 90.46% code coverage and 100% functional coverage. More specifically, the code-coverage report listed “Total Coverage By Instance (filtered view): 90.46%,” while the functional-coverage report listed “Total Coverage By Instance (filtered view): 100.00%.”

```
[PASS] sb_sendall_test (Run 1/10) | Seed: 436110965 | Time: 12s
[PASS] sb_sendall_test (Run 2/10) | Seed: 452901465 | Time: 8s
[PASS] sb_sendall_test (Run 3/10) | Seed: 329161089 | Time: 9s
[PASS] sb_sendall_test (Run 4/10) | Seed: 1037998693 | Time: 9s
[PASS] sb_sendall_test (Run 5/10) | Seed: 348043789 | Time: 8s
[PASS] sb_sendall_test (Run 6/10) | Seed: 2025421 | Time: 9s
[PASS] sb_sendall_test (Run 7/10) | Seed: 1157529901 | Time: 9s
[PASS] sb_sendall_test (Run 8/10) | Seed: 591329261 | Time: 9s
[PASS] sb_sendall_test (Run 9/10) | Seed: 285123085 | Time: 9s
[PASS] sb_sendall_test (Run 10/10) | Seed: 1495177469 | Time: 9s
[PASS] sb_rand_test (Run 1/10) | Seed: 616475765 | Time: 32s
[PASS] sb_rand_test (Run 2/10) | Seed: 1530851705 | Time: 31s
[PASS] sb_rand_test (Run 3/10) | Seed: 2034139613 | Time: 31s
[PASS] sb_rand_test (Run 4/10) | Seed: 987661565 | Time: 31s
[PASS] sb_rand_test (Run 5/10) | Seed: 1719584437 | Time: 35s
[PASS] sb_rand_test (Run 6/10) | Seed: 1235131717 | Time: 37s
[PASS] sb_rand_test (Run 7/10) | Seed: 2136919425 | Time: 40s
[PASS] sb_rand_test (Run 8/10) | Seed: 1975449697 | Time: 261s
[PASS] sb_rand_test (Run 9/10) | Seed: 1216258281 | Time: 63s
[PASS] sb_rand_test (Run 10/10) | Seed: 786385445 | Time: 52s
[PASS] sb_conc_test (Run 1/10) | Seed: 1973359465 | Time: 44s
[PASS] sb_conc_test (Run 2/10) | Seed: 1975460673 | Time: 40s
[PASS] sb_conc_test (Run 3/10) | Seed: 507462929 | Time: 47s
[PASS] sb_conc_test (Run 4/10) | Seed: 1010770881 | Time: 368s
[PASS] sb_conc_test (Run 5/10) | Seed: 463430865 | Time: 34s
[PASS] sb_conc_test (Run 6/10) | Seed: 1142884413 | Time: 38s
[PASS] sb_conc_test (Run 7/10) | Seed: 1145003153 | Time: 44s
[PASS] sb_conc_test (Run 8/10) | Seed: 1914665077 | Time: 39s
[PASS] sb_conc_test (Run 9/10) | Seed: 478091233 | Time: 35s
[PASS] sb_conc_test (Run 10/10) | Seed: 513738465 | Time: 34s
```

Figure 1.3: Regression summary showing the major Sideband tests passing across repeated runs.

The coverage numbers should be interpreted together with the deliberate exclusions documented in the simulation scripts. Some toggle points were excluded because they cor-

respond to fixed protocol constants, reserved fields, or assertion logic that should not be counted as RTL implementation coverage. In addition, selected unreachable toggle cases were excluded where the current legal stimulus architecture cannot drive those conditions meaningfully. The code-coverage review also helped identify redundant RTL logic consisting of two lines that were not functionally used, and these lines were subsequently removed.

Together, the passing assertions, transaction matches, targeted bug discoveries, and final regression results demonstrate that the environment is effective both as a bug-finding platform and as a trustworthy checker for the currently supported Sideband functionality. With the environment architecture, reset methodology, and regression strategy established, the verification effort is well positioned to scale further as additional protocol paths such as RDI are integrated.

Bibliography

- [1] B. Hunter, B. Chen, and R. Lipon, “Reset testing made simple with uvm phases,” in *Proceedings of the Synopsys Users Group (SNUG)*, (San Jose, CA, USA), March 2013.
- [2] C. E. Cummings, “Ovm/uvm scoreboards - fundamental architectures,” in *SNUG (Synopsys Users Group)*, 2013. Rev 1.1.
- [3] J. Rensch and N. Johnson, “How do you verify your verification components?,” 2016. Whitepaper / Conference Paper.